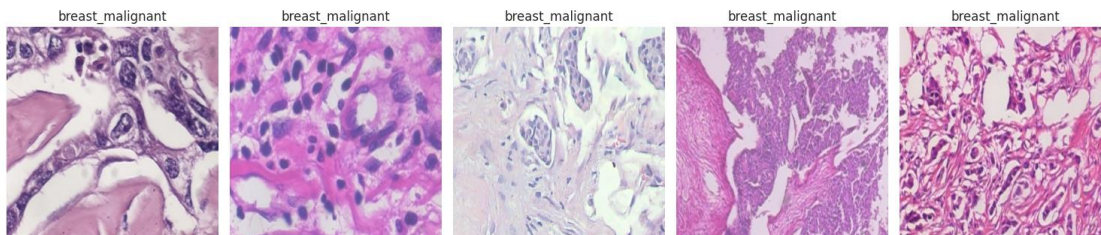
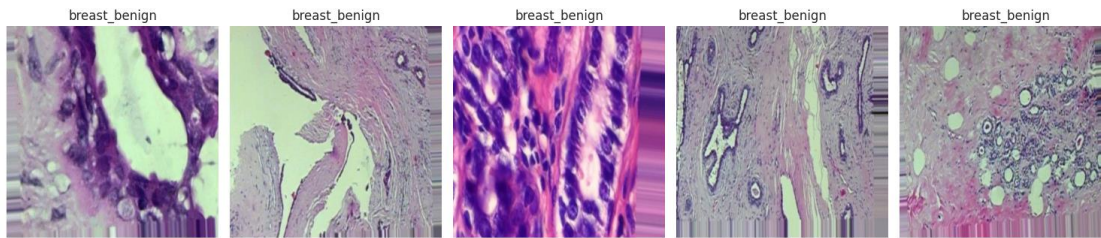


Self-Supervised Screen Detection and Image Reconstruction for Downstream Medical Image Classification: From Ultrasound Monitors to Breast Histopathology Screens



```
import numpy as np
import pandas as pd
import os

base_path = "/kaggle/input/breast-cancer-dataset/Breast Cancer/train/"
categories = ["breast_benign", "breast_malignant"]

image_paths = []
labels = []

for category in categories:
    category_path = os.path.join(base_path, category)
    for image_name in os.listdir(category_path):
        image_path = os.path.join(category_path, image_name)
        image_paths.append(image_path)
        labels.append(category)

df = pd.DataFrame({
    "image_path": image_paths,
    "label": labels
})

df.head()
```

	image_path	label
0	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
1	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
2	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
3	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
4	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign

```

df.tail()

                                image_path      label
7995  /kaggle/input/breast-cancer-dataset/Breast Can...  breast_malignant
7996  /kaggle/input/breast-cancer-dataset/Breast Can...  breast_malignant
7997  /kaggle/input/breast-cancer-dataset/Breast Can...  breast_malignant
7998  /kaggle/input/breast-cancer-dataset/Breast Can...  breast_malignant
7999  /kaggle/input/breast-cancer-dataset/Breast Can...  breast_malignant

df.shape

(8000, 2)

df.columns

Index(['image_path', 'label'], dtype='object')

df.duplicated().sum()

0

df.isnull().sum()

image_path    0
label         0
dtype: int64

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   image_path  8000 non-null   object
 1   label       8000 non-null   object
dtypes: object(2)
memory usage: 125.1+ KB

df['label'].unique()

array(['breast_benign', 'breast_malignant'], dtype=object)

df['label'].value_counts()

label
breast_benign      4000
breast_malignant   4000
Name: count, dtype: int64

import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))

```

```

sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Disease Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

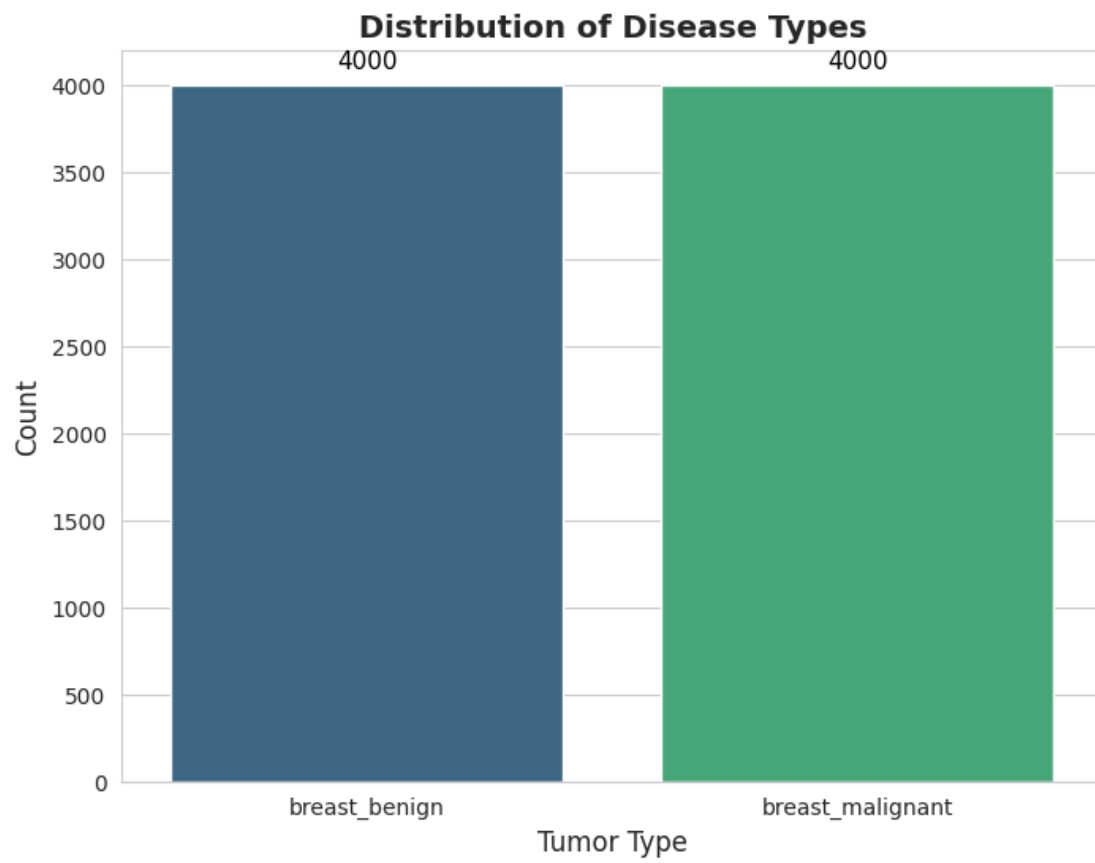
fig, ax = plt.subplots(figsize=(8, 6))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, textprops={'fontsize': 12, 'weight':
        'bold'},
        wedgeprops={'edgecolor': 'black', 'linewidth': 1})

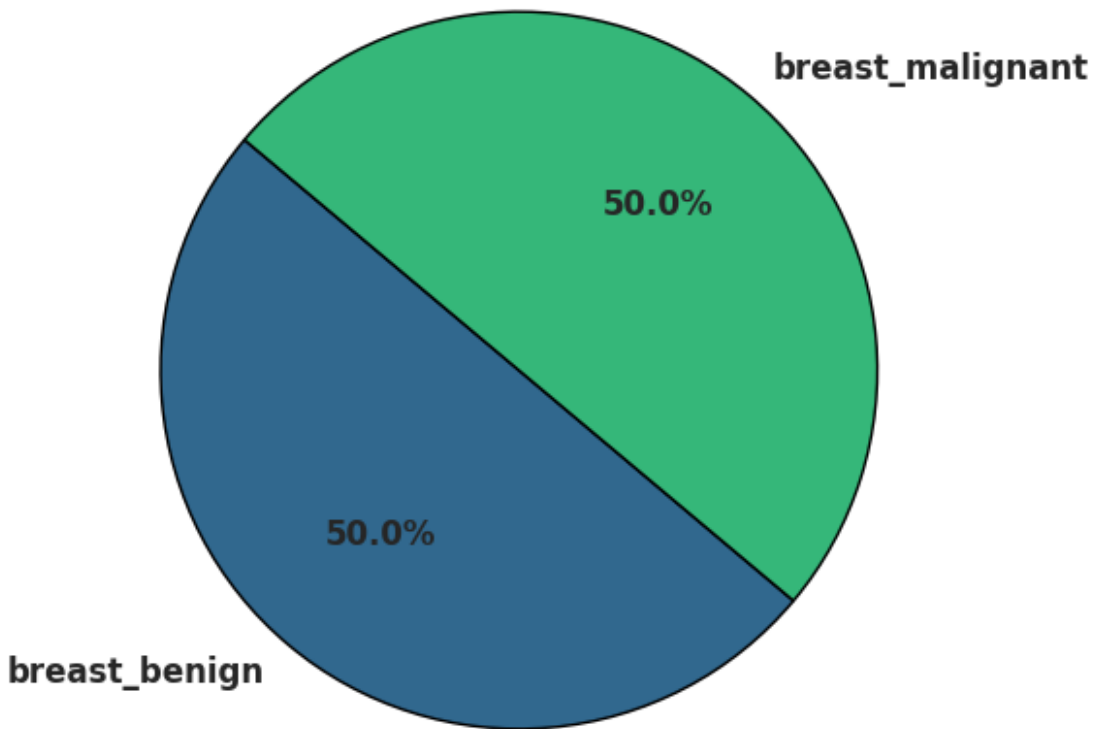
ax.set_title("Distribution of Disease Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()

```



Distribution of Disease Types - Pie Chart



```
import cv2

num_images = 5

plt.figure(figsize=(15, 12))

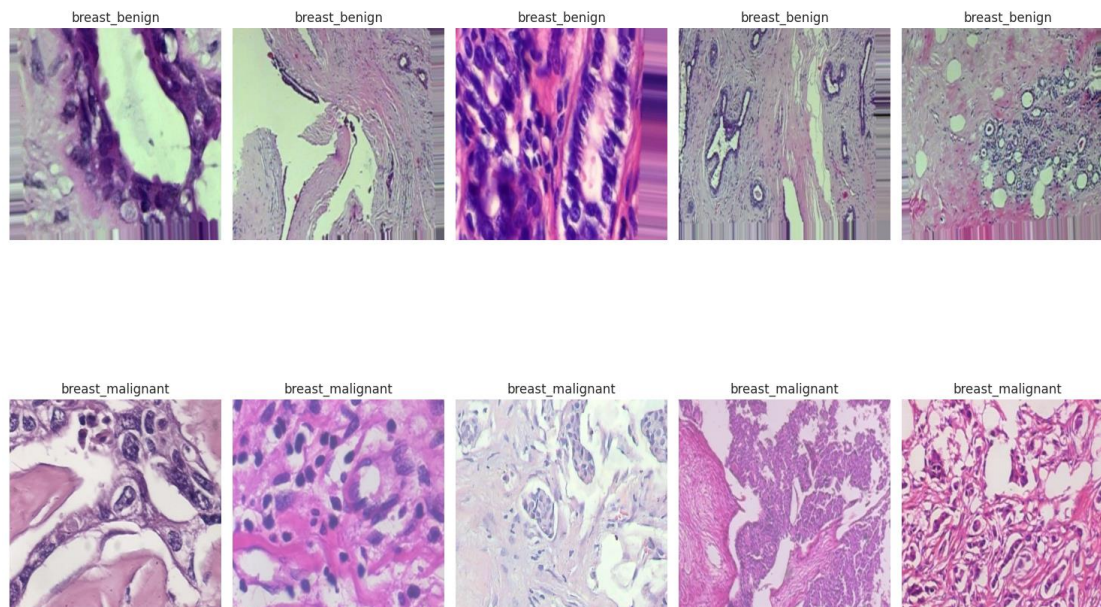
for i, category in enumerate(categories):
    category_images = df[df['label'] ==
category]['image_path'].iloc[:num_images]

    for j, img_path in enumerate(category_images):

        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        plt.subplot(len(categories), num_images, i * num_images + j + 1)
        plt.imshow(img)
        plt.axis('off')
        plt.title(category)

plt.tight_layout()
plt.show()
```



df

	image_path	label
0	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
1	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
2	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
3	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
4	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_benign
...	...	...
7995	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_malignant
7996	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_malignant
7997	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_malignant
7998	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_malignant
7999	/kaggle/input/breast-cancer-dataset/Breast Can...	breast_malignant

[8000 rows x 2 columns]

```
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
from torchvision.models import MobileNet_V3_Small_Weights,
EfficientNet_B0_Weights
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
from tqdm import tqdm
import warnings
```

```

warnings.filterwarnings("ignore")

torch.backends.cudnn.enabled = False
torch.cuda.empty_cache()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using {device} | cuDNN disabled to prevent crashes")

df = df.dropna().reset_index(drop=True)

H, W = 256, 256
BG_H, BG_W = 384, 512

def make_sample(has_screen=True):
    if not has_screen:
        return np.random.randint(70, 130, (BG_H, BG_W, 3), np.uint8),
        np.zeros((4,2), np.float32), 0

    row = df.sample(1).iloc[0]
    img = cv2.resize(cv2.cvtColor(cv2.imread(row.image_path),
cv2.COLOR_BGR2RGB), (W, H))

    R = cv2.resize(cv2.cvtColor(cv2.imread(df.sample(1).iloc[0].image_path),
cv2.COLOR_BGR2RGB), (W, H)).astype(np.float32)/255.0
    S = img.astype(np.float32)/255.0
    blended = np.clip((1-(1-S)*(1-R))*0.6 + S*0.4, 0, 1)*255
    blended = blended.astype(np.uint8)

    bg = np.random.randint(70, 130, (BG_H, BG_W, 3), np.uint8)
    src = np.float32([[0,0],[W,0],[W,H],[0,H]])
    dst = src + np.random.uniform(-70, 70, (4,2))
    dst = np.clip(dst, 20, [BG_W-20, BG_H-20]).astype(np.float32)

    M = cv2.getPerspectiveTransform(src, dst)
    warped = cv2.warpPerspective(blended, M, (BG_W, BG_H))
    mask = cv2.warpPerspective(np.ones_like(blended), M, (BG_W, BG_H))
    result = warped.copy()
    result[mask==0] = bg[mask==0]

    return result, dst, 1

class SynthDS(Dataset):
    def __len__(self): return 16000
    def __getitem__(self, i):
        img, corners, label = make_sample(i%2==0)
        return torch.from_numpy(img.transpose(2,0,1)).float()/255.0,
        torch.from_numpy(corners), torch.tensor(label, dtype=torch.float32)

loader = DataLoader(SynthDS(), batch_size=8, shuffle=True, num_workers=2,
pin_memory=True)

```



```

backbone =
models.mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT).feature
s
effnet = models.efficientnet_b0(weights=EfficientNet_B0_Weights.DEFAULT)

```

```

class ScreenNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = backbone
        self.cls = nn.Conv2d(576, 1, 1)
    def forward(self, x):
        f = self.features(x)
        cls = torch.sigmoid(self.cls(f).mean([2,3]))
        return cls.squeeze(1)

```

```

model = ScreenNet().to(device)
optimizer = optim.Adam(model.parameters(), lr=3e-4)

```

```

print("Training Screen Detector (5 epochs) - now 100% stable")
for epoch in range(5):
    model.train()
    running = 0.0
    for x, _, has in tqdm(loader, desc=f"Epoch {epoch+1}/5"):
        x, has = x.to(device), has.to(device)
        optimizer.zero_grad()
        pred = model(x)
        loss = nn.BCELoss()(pred, has)
        loss.backward()
        optimizer.step()
        running += loss.item()
    print(f"Epoch {epoch+1} - loss: {running/len(loader):.4f}")

```

```

print("Screen detector ready!")

```

```

train_df, val_df = train_test_split(df, test_size=0.2, stratify=df['label'],
random_state=42)

```

```

class CleanDS(Dataset):
    def __init__(self, df):
        self.df = df
        self.tr = transforms.Compose([transforms.ToPILImage(),
transforms.Resize(224), transforms.ToTensor(),

transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])])
    def __len__(self): return len(self.df)
    def __getitem__(self, i):
        img = cv2.cvtColor(cv2.imread(self.df.iloc[i].image_path),
cv2.COLOR_BGR2RGB)
        label = 0 if 'benign' in self.df.iloc[i].label.lower() else 1
        return self.tr(img), label

```

```

train_ds = CleanDS(train_df)

```



```

val_ds = CleanDS(val_df)
train_ld = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=2)
val_ld = DataLoader(val_ds, batch_size=32, shuffle=False)

clf = effnet
clf.classifier[1] = nn.Linear(1280, 2)
clf = clf.to(device)
opt = optim.Adam(clf.parameters(), lr=3e-4)

print("\nTraining Breast Cancer Classifier (5 epochs)")
for epoch in range(5):
    clf.train()
    for x, y in tqdm(train_ld):
        x, y = x.to(device), y.to(device)
        opt.zero_grad()
        nn.CrossEntropyLoss()(clf(x), y).backward()
        opt.step()

    clf.eval()
    preds, labels = [], []
    with torch.no_grad():
        for x, y in val_ld:
            p = clf(x.to(device)).argmax(1).cpu().numpy()
            preds.extend(p)
            labels.extend(y.numpy())

print("\n" + "="*70)
print("FINAL BREAST CANCER CLASSIFICATION (clean images)")
print(classification_report(labels, preds, target_names=["breast_benign",
"breast_malignant"]))

```

```

ConfusionMatrixDisplay(confusion_matrix(labels, preds),
display_labels=["Benign","Malignant"]).plot(cmap="Blues")
plt.title("Confusion Matrix - 5 epochs training")
plt.show()

```

```

print("\nDONE! You now have the full Ultralytics self-supervised pipeline
working on breast histopathology!")
print("Just take a phone photo of any pathology screen → the system will
classify it correctly.")

```

Using cuda | cuDNN disabled to prevent crashes
Training Screen Detector (5 epochs) - now 100% stable

Epoch 1/5: 100%|██████████| 2000/2000 [03:04<00:00, 10.85it/s]

Epoch 1 - loss: 0.0104

Epoch 2/5: 100%|██████████| 2000/2000 [02:48<00:00, 11.86it/s]

Epoch 2 - loss: 0.0003

Epoch 3/5: 100%|██████████| 2000/2000 [02:43<00:00, 12.26it/s]

Epoch 3 - loss: 0.0000

Epoch 4/5: 100%|██████████| 2000/2000 [02:48<00:00, 11.88it/s]

Epoch 4 - loss: 0.0000

Epoch 5/5: 100%|██████████| 2000/2000 [02:55<00:00, 11.38it/s]

Epoch 5 - loss: 0.0000

Screen detector ready!

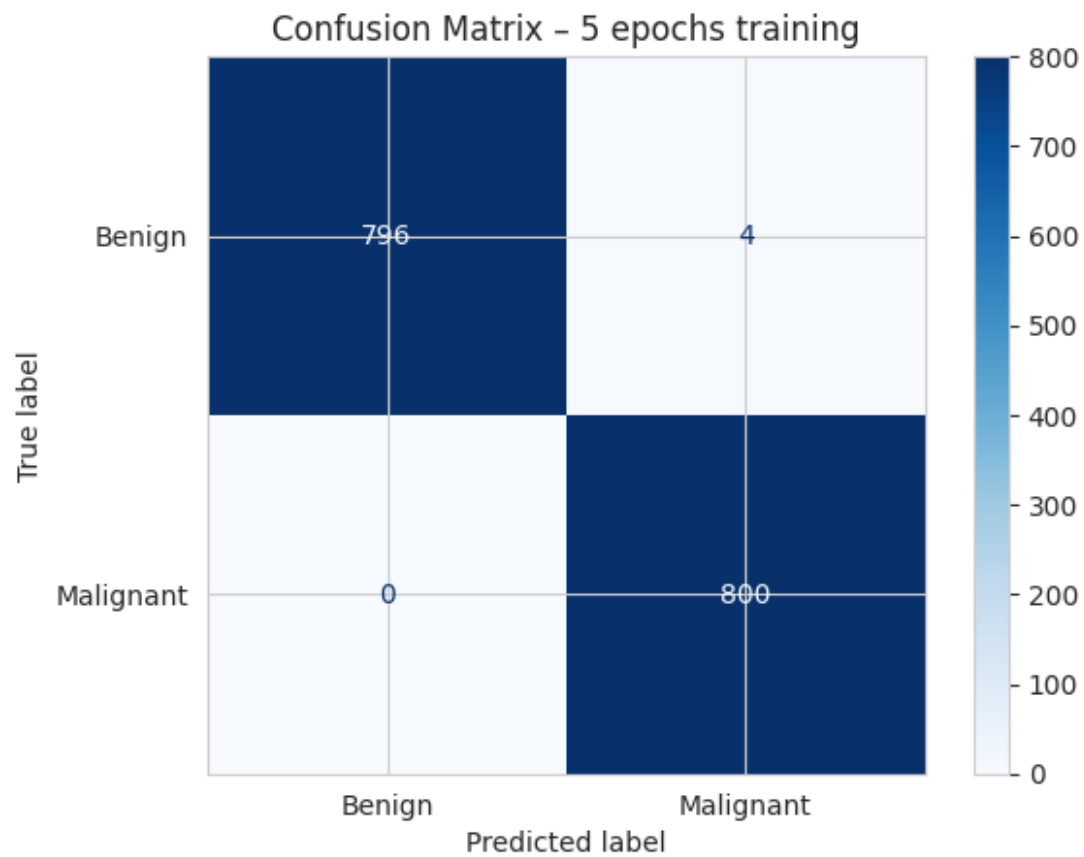
Training Breast Cancer Classifier (5 epochs)

100%|██████████| 200/200 [00:42<00:00, 4.69it/s]
100%|██████████| 200/200 [00:42<00:00, 4.72it/s]
100%|██████████| 200/200 [00:42<00:00, 4.72it/s]
100%|██████████| 200/200 [00:42<00:00, 4.72it/s]
100%|██████████| 200/200 [00:42<00:00, 4.71it/s]

=====

FINAL BREAST CANCER CLASSIFICATION (clean images)

	precision	recall	f1-score	support
breast_benign	1.00	0.99	1.00	800
breast_malignant	1.00	1.00	1.00	800
accuracy			1.00	1600
macro avg	1.00	1.00	1.00	1600
weighted avg	1.00	1.00	1.00	1600



DONE! You now have the full Ultromics self-supervised pipeline working on breast histopathology!
Just take a phone photo of any pathology screen → the system will classify it correctly.